2.d Idioms

Writing Idiomatic Kotlin Code

Contents

1 **Data Classes & Functions**
DTOs, default arguments, named parameters & string templates

2 **Null Safety & Collections**
Safe calls, Elvis operator, read-only lists & maps, filtering

3 **Scope Functions & Expressions**
let, apply, with, also, run — plus when, try, if as expressions

4 **Practice Exercises**
Solved and unsolved exercises applying Kotlin idioms

What Are Idioms?

Idioms are commonly used patterns and conventions that make Kotlin code more concise, readable, and expressive. They leverage Kotlin's language features to replace verbose Java-style boilerplate.

Key idiom categories from the official docs:

Data Classes

Auto-generate getters, equals(), hashCode(), toString(), copy() and componentN().

Null Safety

Safe calls (?.), Elvis (?:), let for null checks, and safe casts (as?).

Scope Functions

let, apply, with, also, run — configure objects and execute blocks.

Expressions

when, if, try-catch as expressions that return values.

Data Classes & Default Arguments

Creating DTOs (Data Transfer Objects)

```
data class Customer(
    val name: String,
    val email: String
)
```

Auto-generated:

- Getters for all properties
- equals() / hashCode()
- toString() — "Customer(name=..., email=...)"

Default & Named Arguments

```
fun greet(
    name: String,
    greeting: String = "Hello"
) = "$greeting, $name!"

println(greet("Alice"))
```

Output: Hello, Alice!

Key Points

- Eliminates need for overloaded methods
- Named args improve readability at call sites

String Templates & Collections

String Templates

```kotlin
val name = "Kotlin"
println("Hello, $name!")
println("Length: ${name.length}")
```

Output:
Hello, Kotlin!
Length: 6

Read-Only Collections

```kotlin
val list = listOf("a", "b", "c")
val map = mapOf("name" to "Alice")

// Filtering a list
println(list.filter { it != "b" })
// Accessing a map
println(map["name"])
```

Output:
[a, c]
Alice

Ranges & Iteration

```kotlin
for (i in 1..100) { }          // closed range, includes 100
for (i in 1 until 100) { }     // half-open, excludes 100
for (x in 2..10 step 2) { }    // step by 2
```

Null Safety Idioms

Kotlin's type system eliminates NullPointerException at compile time

```kotlin
// Safe call operator (?.)
val files: List<String>? =
    listOf("a.kt", "b.kt", "c.kt")
println(files?.size)

// Elvis operator (?:)
println(files?.size ?: "empty")

// let with non-null value
files?.let { println(it) }

// let with null value
val empty: List<String>? = null
empty?.let { println(it) }
println("empty is $empty")
```

Null Safety Operators

- `?.` Safe call: returns null if receiver is null
- `?:` Elvis: provides default when left side is null
- `?.let{}` Execute block only if value is not null
- `as?` Safe cast: returns null if cast fails

Output:

```
3
3
[a.kt, b.kt, c.kt]
empty is null
```

Scope Functions

let

Object as argument
(it)

```
val len = str?.let
{
    println(it)
    it.length
}
```

apply

Object as receiver
(this)
Returns the object

```
val person =
Person().apply {
    name = "Alice"
    age = 25
}
```

with

Object as receiver
(this)
Returns lambda
result

```
with(config) {
    width = 100
    height = 200
}
```

also

Object as argument
(it)
Returns the object

```
val numbers =

mutableListOf(1,8)
  .also {
    println(it)
  }
```

run

Object as receiver
(this)
Returns lambda
result

```
val result =
  service.run {
    port = 8080
    query()
}
```

Control Flow as Expressions

when Expression

```
fun transform(color: String)
  = when (color) {
    "Red"   -> 0
    "Green" -> 1
    "Blue"  -> 2
    else    -> -1
  }

// Using transform
println(transform("Green"))
```

Output: 1

try-catch Expression

```
// try-catch as expression
val input = "42"
val number = try {
    input.toInt()
} catch (e: Exception) {
    -1
}

println(number)
```

Output: 42

if Expression

```
val x = 1
val y = if (x == 1) {
    "one"
} else if (x == 2) {
    "two"
} else {
    "other"
}

println(y)
```

Output: one

Key Insight In Kotlin, when, if, and try-catch are expressions — they return values. No need for separate return statements!

More Useful Idioms

Extension Functions

```
fun String.lastChar(): Char
  = this[length - 1]

println("Kotlin".lastChar())
```

Output:
n

Singleton

```
object Logger {
  fun log(msg: String) {
    println(msg)
  }
}

Logger.log("Hello!")
```

Output:
Hello!

Lazy Initialization

```
val lazyValue: String by lazy {
  println("Computed once")
  "Hello, Kotlin"
}

println(lazyValue)
println(lazyValue)
```

Output:
Computed once
Hello, Kotlin
Hello, Kotlin

TODO() Function

```
fun calculate(): Int {
  TODO("Not implemented")
}

try { calculate() }
catch (e: Exception) {
  println(e.message)
}
```

Output:
Not implemented

Practice Exercises

Exercise 1: Data Class & apply

Create a data class Book with title, author, and year. Use apply to create an instance and print its details using string tem plates.

Solution

```
data class Book(val title: String, val author: String, val year: Int)

fun main() {
    val book = Book("Kotlin in Action", "Isakova", 2017)
    book.also { println("${it.title} by ${it.author} (${it.year})") }
}
```

Output: Kotlin in Action by Isakova (2017)

Concepts Used: Data class • also scope function • String templates • Destructuring

Exercise 2: Null Safety & when

Write a function that takes a nullable String, uses the Elvis operator for a default, and a when expression to return a message.

Solution

```
fun greetUser(name: String?): String {
    val user = name ?: "Guest"
    return when {
        user == "Guest"    -> "Welcome, stranger!"
        user.length > 10   -> "Hello, $user! (long name)"
        else               -> "Hi, $user!"
    }
}

// Three branches in action
println(greetUser("Alice"))
println(greetUser(null))
println(greetUser("Christopher"))
```

Output:

```
Hi, Alice!
Welcome, stranger!
Hello, Christopher! (long name)
```

Concepts Used: Nullable types • Elvis operator (?:) • when without argument • String templates • Smart casts

Exercise 3: Extension Function

Write an extension function on List<Int> that returns the second-largest element, or null if the list is too short. Use safe calls.

Solution

```
fun List<Int>.secondLargest(): Int? =
    this.distinct().sortedDescending().getOrNull(1)

fun main() {
    val nums = listOf(3, 7, 7, 1, 5)
    println(nums.secondLargest())  // 5
    println(listOf(42).secondLargest())  // null
}
```

Output:
5
null

Concepts Used: Extension function • distinct() • sortedDescending() • getOrNull() • Nullable return type

Exercise 4: when as Expression

Write a function describe(obj: Any): String using when with type checks (is), ranges (in), and an else branch.

Solution

```kotlin
fun describe(obj: Any): String = when (obj) {
    1             -> "One"
    "hello"       -> "Greeting"
    is Long       -> "Long: $obj"
    is String     -> "String of length ${obj.length}"
    is Int -> when {
        obj in 1..10 -> "Small int: $obj"
        else         -> "Big int: $obj"
    }
    else          -> "Unknown: $obj"
}

// Five branches in action
println(describe(1))
println(describe("hello"))
println(describe(1000L))
println(describe("Kotlin"))
```

Output:

```
One
Greeting
Long: 1000
String of length 6
Big int: 42
```

Concepts: when expr. • Type checks (is) • Ranges (in) • Smart casts • Nested when

Your Turn — Practice Exercises

1 Scope Function Choice

Given a class Person(var name: String, var age: Int), use apply, also, and let to create and log an instance. Explain when each is preferred.

Hint: Think about what each scope function returns.

2 Collection Pipeline

Create a list of 10 numbers. Use filter, map, and firstOrNull to find the first even number greater than 5, doubled. Handle the null case.

Hint: Chain operations and use the Elvis operator.

3 Extension Function

Write an extension function on List<Int> that returns the second-largest element, or null if the list is too short. Use safe calls.

Hint: Use distinct(), sortedDescending(), and getOrNull().

4 when as Expression

Write a function describe(obj: Any): String using when with type checks (is), ranges (in), and an else branch.

Hint: when can check types with 'is' keyword.

Challenge Exercises

5 Idiomatic Refactoring

Take a Java-style Kotlin function with null checks, manual loops, and verbose if/else. Refactor it using safe calls, let, when, and filter/map.

Hint: Replace Java patterns one by one.

6 Builder with apply

Create a DSL-like builder for an HTML element using data classes, apply, and extension functions. Build a simple tag tree.

Hint: Combine apply with nested lambdas.

💡 Tips for Success

Experiment
Try things in the Kotlin Playground — break code to learn!

Read the Docs
kotlinlang.org/docs/idioms is your best reference

Compare
Put Java and Kotlin solutions side by side

Readability First
Don't overuse idioms — clear code beats clever code

Practice Daily
Solve Advent of Code challenges in Kotlin

Refactor
Review existing code and find idiom opportunities